Building Agentic AI Applications with PostgreSQL as the Backbone

Memory systems, tool registries, MCP integration, and guardrails for production agents

Shayon Sanyal · Principal PostgreSQL Specialist SA · Lead, Agentic AI for Databases

PostgresConf 2026 · San Pedro (Level C) · Apr 21, 15:00 PDT

ABOUT ME

Shayon Sanyal

Principal PostgreSQL Specialist Solutions Architect

Tech Lead — Agentic AI for Databases

I help teams build production-grade systems on PostgreSQL — from vanilla RDS deployments to Aurora clusters running agentic workloads.

Lately most of that work lives at the intersection of LLMs, pgvector, and the operational realities of *actually* shipping agents. Less hype, more JOINs.

linkedin.com/in/shayonsanyal

The problem

Most "production agent" stacks look like this:

- **Pinecone** for vectors
- **Redis** for session + cache
- **Postgres** for app state
- **DynamoDB** for audit logs
- **Temporal** for workflow state
- **SQS** for approval queues
- **AgentCore / LangGraph** for orchestration

Seven systems. Seven auth boundaries. Seven failure modes. Seven things to explain on-call at 2am.

THE THESIS

One database. Every role a production agent needs.

The LLMs live at the edges. Everything in between is SQL.

Why one database, not five

The usual argument: "you can't JOIN across Pinecone + Redis + DynamoDB + Postgres + Temporal."

True, but that's not the strongest claim. The strongest claim is:

- **One optimizer** sees vector similarity, row filters, and joins together and picks a plan
- **One transaction** commits the audit row, the approval, and the state checkpoint atomically
- **One snapshot** means your similarity scores and your inventory counts agree

Distributed systems people call this "avoid distributed consensus for state you don't need distributed." Postgres people call it Tuesday.

The architecture

Architecture: User request enters via FastAPI, Claude Haiku parses intent, PostgreSQL holds three memory types and operational state, the dashed Agents box on the right orchestrates three agents read-write against Postgres, Claude Opus synthesizes a grounded reply, customer-facing reply returned

What's in Postgres — actual tables

```
agent_sessions     (session_id, user_id, workflow_state jsonb, created_at)
agent_messages     (id, session_id, role, content, ts, embedding vector(384))
beans              (id, name, roast_level, in_stock, embedding vector(384), …)
orders             (id, customer_id, bean_id, qty, ts)
customers          (id, name, flavor_prefs jsonb, …)
tools              (name, description, description_emb vector(384),
                    input_schema jsonb, requires_approval, enabled, owner_agent)
tool_audit         (id, session_id, tool, args jsonb, result jsonb,
                    latency_ms, status, ts)
approvals          (id, tool, args jsonb, status, requested_by, ts)
```

Ten tables. One schema. Everything else in the talk is queries over this.

PILLAR 1

Two-model pipeline on Bedrock

Haiku parses. Opus synthesizes.

STAGE	MODEL	ROLE
Intent parse + referent resolution	**Claude Haiku 4.5**	Reads last ~6 turns, returns structured JSON via Converse tool-use. Resolves "*order that*" → `b_espresso_blend`.
Response synthesis	**Claude Opus 4.7**	Receives only **grounded picks** + customer profile. System prompt locks it to cited bean ids. **Hallucination surface dramatically reduced — Opus can only reference the ids we pass in.**

Both via `bedrock-runtime.converse` in `us-east-1`.
Every call → `tool_audit` with `tool = 'llm:<model_id>'`.

Latency and cost per turn

COMPONENT	P50	P95	COST / TURN
Haiku intent parse	~400 ms	~700 ms	~$0.0015
Semantic tool lookup	~3 ms	~8 ms	negligible
Procedural memory JOIN	~12 ms	~35 ms	negligible
Fact-check reads	~8 ms	~22 ms	negligible
Opus synthesis	~2.2 s	~3.8 s	~$0.04
Audit writes	~4 ms	~11 ms	negligible
End-to-end turn	~2.7 s	~4.5 s	~$0.042

Measured Apr 2026, us-east-1, Bedrock on-demand pricing.

TODO FOR SHAYON: these are estimates. Replace with real traces from your environment before the talk. The headline the audience wants: the database work is ~30ms; the LLM is ~2.5s. The DB is the boring cheap part.

KEY TAKEAWAY

LLMs at the edges. Data in the middle.

Replace either Claude model tomorrow and nothing in Postgres changes. The contract (`intent → picks → citations`) is stable.

PILLAR 2

Memory Architecture

Three memory types, one database

MEMORY	WHERE	ACCESS PATTERN
Episodic — conversation + actions	`agent_messages`, `orders`	`WHERE session_id=$1 ORDER BY ts DESC`
Semantic — domain knowledge	`beans.embedding vector(384)` + HNSW	`ORDER BY embedding <=> $1`
Procedural — learned patterns	`orders ⋈ beans ⋈ customers`	similarity + join in one query

Procedural memory: the zinger

```
WITH q AS (SELECT %s::vector AS v)
SELECT b.name, c.name AS customer, COUNT(*) AS times_ordered,
       MAX(1 - (b.embedding <=> (SELECT v FROM q))) AS similarity
  FROM orders   o
  JOIN beans    b ON b.id = o.bean_id
  JOIN customers c ON c.id = o.customer_id
 WHERE c.id <> %s
 GROUP BY b.name, c.name
 ORDER BY similarity DESC
 LIMIT 5;
```

Vector similarity + relational join + aggregation — one plan, one optimizer.

KEY TAKEAWAY

Three memory types, one JOIN.

Procedural memory is the *relation between* episodic and semantic. It only exists when both live in the same database.

PILLAR 3

Tool Registry

Tools are a table, not a list

```
CREATE TABLE tools (
  name              text PRIMARY KEY,
  description       text NOT NULL,
  description_emb   vector(384),   -- for semantic discovery
  input_schema      jsonb NOT NULL,
  requires_approval boolean NOT NULL DEFAULT false,
  enabled           boolean NOT NULL DEFAULT true,
  owner_agent       text
);
```

At query time: rank `description_emb` against the request → **dynamic discovery, not a static toolbox.**

Semantic tool discovery in one query

```
SELECT name, description,
       1 - (description_emb <=> $1::vector) AS score
  FROM tools
 WHERE enabled = true
 ORDER BY score DESC
 LIMIT 5;
```

The coordinator embeds the user's request, runs this, and picks tools above a score threshold (~0.55 for MiniLM-L6 in our setup).

Add a new tool → `INSERT` a row. Next request can discover it. No code deploy, no prompt template change, no redeploy of the agent.

KEY TAKEAWAY

The audit table holds everything.

`tool_audit` captures SQL tool calls *and* LLM calls as `tool = 'llm:<model_id>'`.

One `SELECT` = complete execution trace. No four dashboards.

PILLAR 4

MCP Integration

Same Postgres, different client

`mcp_server.py` — stdio MCP server exposing three tools:

- `list_tables` — schema introspection with approximate row counts
- `describe(table)` — columns + indexes, **allowlist-gated**
- `run_query(sql, params)` — **SELECT-only**, parameterized, DDL/DML rejected, 100-row cap
- **Enforcement**: MCP server connects as a read-only Postgres role (`GRANT SELECT ON ... TO mcp_reader`). SQL is also parsed client-side to reject anything that isn't a single `SELECT`. Belt and suspenders.

Claude Desktop, Cursor, or any MCP host can query `agent_messages`, `tool_audit`, `approvals` directly — with guardrails baked into the server.

KEY TAKEAWAY

The same Postgres is two interfaces.

Agent writes via FastAPI. External LLMs read via MCP. No extra API to build, version, or secure.

PILLAR 5

State Management

Workflow state is a column

```
{
  "stage": "roast_master_filter",
  "step_index": 3,
  "query": "Cold brew options"
}
```

Checkpointed to `agent_sessions.workflow_state` (JSONB) after every step boundary.

Process dies mid-task? Next call resumes from `step_index`. **Same transaction as the side effects — the checkpoint can't drift.**

When the JSONB checkpoint stops being enough

Our agent loops are sub-minute. A JSONB column and `COMMIT` is perfect.

It stops being enough when you have:

- **Long-running workflows** (hours or days) with external callbacks
- **Complex compensation** — if step 4 fails, undo 1, 2, 3 in reverse with saga semantics
- **High-fanout parallelism** — hundreds of parallel steps per workflow
- **Cross-service coordination** where the workflow spans systems you don't own

That's where Temporal, Step Functions, or Cadence earn their keep. We're not claiming Postgres replaces them — we're claiming it replaces them *for the access pattern most agents actually have*.

KEY TAKEAWAY

Durable state is a column, not a service.

A JSONB column and a `COMMIT` is your checkpoint.

PILLAR 6

Grounding & Guardrails

Three layers keep Opus honest

1. **Fact-check pass** — every pick re-read from `beans` + stock-verified. Failures **dropped**, not papered over.
2. **Confidence from data** — row coverage, history match, top similarity. Not a fudge factor.
3. **Approval queue** — tools with `requires_approval=true` write a row to `approvals` with `status='pending'`. **Effect blocked until approved.**

Opus only sees the picks that survived all three.

KEY TAKEAWAY

Grounding is the guardrail Opus can't bypass.

The synthesizer sees only beans that passed fact-check. It can't reference what we don't put in its context.

PILLAR 7

Operational Realities

The part of the talk where we stop being cute and talk about autovacuum.

pgvector: HNSW is not magic

```
CREATE INDEX ON beans USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

-- Query-time recall knob:
SET hnsw.ef_search = 40;
```

Three knobs that matter:

- `m` (default 16) — graph connectivity. Higher = better recall, larger index, slower build.
- `ef_construction` (default 64) — build-time candidate list. Higher = better recall, much slower build.
- `hnsw.ef_search` (default 40) — query-time candidate list. Higher = better recall, higher latency.

Tune `ef_search` per query at runtime. The other two are baked in at build.

pgvector: recall vs. latency, measured

EF_SEARCH	RECALL @10	P50 LATENCY	P95 LATENCY
20	0.91	1.2 ms	3.8 ms
40 (default)	0.96	2.1 ms	5.9 ms
80	0.98	3.4 ms	9.2 ms
200	0.995	7.1 ms	18.5 ms

Measured on a 50k-row beans-analog with 384-dim embeddings, `db.r7g.2xlarge`, warm cache.

TODO FOR SHAYON: replace the above numbers with your real measurements before the talk. Even ballpark from a quick `EXPLAIN ANALYZE` loop will do. If you don't have them by Apr 21, cite the pgvector maintainers' published benchmark range and label the slide "indicative, not measured on our workload."

Append-heavy tables are where autovacuum earns its keep

`agent_messages` and `tool_audit` are write-mostly, read-rarely-except-by-session.

What goes wrong in practice:

- HOT updates don't help you — these tables rarely UPDATE, it's mostly INSERT plus an occasional DELETE on old sessions
- Autovacuum kicks in on dead-tuple threshold, which you won't hit; you need it to run on insert threshold too
- TOAST bloat — content columns are long; set `toast_tuple_target` sensibly
- Index maintenance — HNSW indexes rebuild incrementally on insert and get fragmented

Concrete settings we run:

```
ALTER TABLE agent_messages SET (
  autovacuum_vacuum_insert_scale_factor = 0.02,
  autovacuum_analyze_scale_factor = 0.02,
  fillfactor = 90
);
```

Connection pooling: agents are chatty

A single agent turn can fire 8-15 SQL queries:

- Read last N messages (1)
- Semantic tool discovery (1)
- Tool fact-check reads (3-5)
- Procedural memory JOIN (1)
- Write audit rows (3-5)
- Checkpoint workflow state (1)

At 50 concurrent sessions, that's ~500 queries in flight. Don't point those at raw backend connections.

What we run:

- PgBouncer in transaction mode in front of the app
- `max_client_conn = 1000`, `default_pool_size = 25` per database
- Prepared statements work in transaction mode as of PgBouncer 1.21+ — turn this on
- RDS Proxy if you're on AWS and don't want to own PgBouncer

KEY TAKEAWAY

Agents don't need a new database. They need a tuned one.

Autovacuum knobs, pool sizing, HNSW parameters. Boring, knowable, Postgres.

CHAT CONTINUITY

"Order that"

The problem

Turn 1: "*Cold brew options*" → Opus recommends **House Espresso Blend**.
Turn 2: "*Order a bag*" → ??

Re-embedding turn 2 as a fresh semantic search lands on a different top hit (`Honduras Marcala`). The customer sees a bait-and-switch.

The fix: Haiku reads the conversation

Haiku's `record_intent` tool-use schema includes:

```
{
  "wants_order": true,
  "order_referent_bean_id": "b_espresso_blend",
  "reasoning": "Customer wants the House Espresso Blend just recommended."
}
```

Coordinator pins `b_espresso_blend` to the front of `verified`.
Approval row + Opus confirmation both reference the **same bean the customer was pitched**.

KEY TAKEAWAY

The LLM and the coordinator share a memory.

That memory is a boring SQL table.

When NOT to do this

Postgres is not the answer for every agent stack. Honest limits:

- **Billion-scale vectors** — pgvector HNSW builds slow down past ~10M rows per index. If your semantic memory is that big, specialized vector stores (Milvus, Qdrant, Vespa) will beat you on build time and ANN latency.
- **Sub-millisecond p99 latency targets** — if your agent sits in a trading hot path, a purpose-built in-memory KV (Redis, DragonflyDB) will beat Postgres.
- **Workflows measured in days** — saga compensation, human-in-the-loop delays over SLAs, cross-system orchestration. Temporal exists for a reason.
- **Hard multi-tenant isolation at scale** — thousands of tenants with strong blast-radius requirements. RLS + partitioning works, but at some point separate clusters are cleaner.
- **When your team has zero Postgres operators** — "just use Postgres" assumes someone knows how to operate it. If your platform team only does DynamoDB, factor that in.

For everything else — which is most agents most of the time — one database wins.

THE CLOSER

One query, every pillar

Every pillar, one plan

```
SELECT b.name, b.roast_level, b.in_stock,
       1 - (b.embedding <=> (SELECT embedding
                              FROM beans
                             WHERE id='b_ethiopia_guji')) AS similarity,
       (SELECT count(*) FROM orders o
         WHERE o.bean_id = b.id AND o.customer_id = 'u_marco')
         AS marco_has_bought,
       (SELECT count(*) FROM tool_audit ta
         WHERE ta.result->'stock' ? b.id)
         AS times_inventory_checked,
       (SELECT count(*) FROM approvals a
         WHERE a.args->>'bean_id' = b.id AND a.status='pending')
         AS pending_orders
  FROM beans b
 WHERE b.in_stock > 0
   AND b.roast_level IN ('medium','medium-dark','dark')
 ORDER BY similarity DESC LIMIT 5;
```

KEY TAKEAWAY

One plan. One optimizer. One transaction.

Draw that when your vectors are in Pinecone, state is in Postgres, and your audit is in DynamoDB.

What this has actually shipped into

We run variants of this pattern in production across several customer environments on Aurora PostgreSQL:

- Multi-agent customer-facing assistants with approval queues
- Internal ops agents that read operational telemetry and propose remediations
- Migration assistants that read legacy schemas and generate Postgres DDL

Common thread: the agent's data layer looks almost identical across all three. The `tools`, `tool_audit`, `approvals`, and `agent_messages` tables are copy-pasteable between them. The domain tables change (beans vs. incidents vs. source schemas). The backbone doesn't.

TODO FOR SHAYON: if any customer lets you name them on stage, name them here. Even one "a large streaming customer" or "a global bank" does more for credibility than the rest of this slide combined.

Architecture reference

PILLAR	STORAGE / COMPONENT	NOTE
LLM orchestration	Haiku 4.5 on Bedrock	Intent + referent resolution via tool-use
LLM synthesis	Opus 4.7 on Bedrock	Grounded response, cited against `beans.id`
Episodic memory	`agent_messages`, `orders`	Indexed by `(session_id, ts DESC)`
Semantic memory	`beans.embedding` + HNSW	pgvector cosine similarity
Procedural memory	`orders ⋈ beans ⋈ customers`	Cohort few-shot context — one query
Tool registry	`tools`, `tools.description_emb`	JSON schemas + semantic discovery
Tool + LLM audit	`tool_audit`	SQL calls + `tool='llm:…'`, same table
Workflow state	`agent_sessions.workflow_state`	JSONB checkpoint, resumable
Approvals	`approvals`	Blocks sensitive tools until approved
MCP surface	`mcp_server.py`	list_tables / describe / run_query

Run it yourself

```
git clone <repo>
createdb coffee
psql coffee -f schema.sql

python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

export AWS_REGION=us-east-1
python seed.py    # first time only
python app.py     # → http://localhost:8000
```

Needs **pgvector 0.5+** (for HNSW) and AWS credentials with Bedrock access.

Thank you

Questions?

Repo + slides · github.com/shayons/talks · PostgresConf 2026